

Instituto Tecnológico y de Estudios Superiores de Monterrey

Proyecto Integrador MiBici

Etapa 2: MLflow Project y tracking avanzado

Operaciones de Aprendizaje Automático

Maestría en Inteligencia Artificial Aplicada

Gerardo Iván Conde Reynoso
Matrícula: A01840602

Equipo 32 — Entrega individual autorizada

Profesor: Dr. Iván Reyes Amezcua

25 de octubre de 2026

Índice

Índice de tablas **3**

Índice de figuras **3**

Resumen **4**

1. Alcance de la Etapa 2 **5**

2. Infraestructura reproducible con MLflow **5**

2.1. Tracking server y almacenamiento 5

2.2. MLflow Project 6

2.3. Particiones temporales 7

3. Tracking avanzado, búsquedas y artefactos **8**

3.1. Convención de tracking 8

3.2. Búsquedas con nested runs 9

3.3. Artefactos por candidato 10

4. P1: predicción de demanda **12**

5. P2: clasificación de riesgo **14**

6. P3: segmentación de estaciones **15**

7. Model Registry y ciclo de vida **17**

8. Git, repositorio y reproducibilidad **20**

8.1. Condición de trabajo individual 21

9. Conclusiones **22**

Contrato de propiedad académica y uso responsable de inteligencia artificial **24**

Referencias **25**

Índice de tablas

1. Entrypoints definidos en MLflow Project. 6
2. Particiones temporales utilizadas en la Etapa 2. 8
3. Búsquedas de hiperparámetros registradas como nested runs. 9
4. Artefactos principales registrados por pregunta. 11
5. Comparación final de modelos para P1. 13
6. Comparación final de modelos para P2. 14
7. Comparación final de modelos para P3. 16
8. Modelos registrados y esquema de ciclo de vida. 17
9. Integración de la Etapa 2 mediante Pull Requests. 20

Índice de figuras

1. Nested runs de P3 10
2. Artefactos del modelo P3 12
3. Model Registry 18
4. Linaje del modelo P3 19

Resumen

Esta segunda etapa del proyecto integrador consolida el trabajo previo sobre los datos abiertos de MiBici en un flujo reproducible de MLOps. El objetivo principal ya no consiste únicamente en entrenar modelos, sino en demostrar que la ingesta, experimentación, comparación, selección, trazabilidad y registro de modelos pueden ejecutarse de manera controlada y verificable. Para ello se implementó un `MLproject` con puntos de entrada reproducibles, un servidor local de MLflow con backend SQLite, búsquedas de hiperparámetros con corridas anidadas, registro sistemático de artefactos y un esquema de ciclo de vida basado en MLflow Model Registry.

El proyecto conserva tres preguntas de aprendizaje automático: P1 para estimar demanda diaria por estación, P2 para clasificar riesgo de disponibilidad y P3 para segmentar estaciones según sus patrones de uso. En los tres casos se mantuvieron particiones temporales y TEST se reservó para evaluación final. Las decisiones de selección se sustentaron en VALIDATION o en métricas calculadas exclusivamente sobre TRAIN, evitando utilizar TEST como criterio de elección.

Los modelos seleccionados fueron PoissonRegressor para P1, HistGradientBoostingClassifier tuneado para P2 y K-Means con $k = 4$ para P3. Cada uno fue registrado como versión 1 en MLflow Model Registry y quedó asociado al alias `champion`. El registro conserva linaje hacia la corrida de origen, commit de Git y versión de datos `7aa5e436d887`. MLflow está diseñado precisamente para dar soporte a experimentación, reproducibilidad y administración del ciclo de vida de modelos (MLflow, s.f.-a, s.f.-b; Zaharia et al., 2018).

Resultado central de la etapa

La entrega integra un flujo reproducible de principio a fin: `MLproject` → tracking de experimentos → búsqueda anidada → selección justificada → artefactos y linaje → Model Registry con alias `champion`.

1. Alcance de la Etapa 2

El proyecto utiliza datos abiertos del sistema MiBici, publicados en archivos mensuales por el propio sistema de bicicleta pública del Área Metropolitana de Guadalajara (MiBici, s.f.). La segunda etapa reutiliza las preguntas, variables y particiones temporales definidas anteriormente, pero concentra la evaluación en reproducibilidad, comparación experimental, trazabilidad y administración del ciclo de vida de modelos.

El repositorio utilizado para esta versión es:

https://github.com/GIvanConder/TC5061-Proyecto-MiBici-Gerardo-Ivan-Conde-Reynoso

El experimento central de MLflow se denomina `mibici-equipo32`. La versión de datos registrada en los experimentos finales es `7aa5e436d887`. Los datos crudos, datos procesados, modelos serializados, `mlflow.db`, `mlruns/` y `mlartifacts/` se excluyen de Git para evitar versionar archivos reconstruibles o pesados.

MLflow Projects proporciona un formato estándar para empaquetar código reproducible mediante un archivo `MLproject`, puntos de entrada parametrizados y una definición del entorno (MLflow, s.f.-c). Esta capacidad se utilizó como mecanismo común para ingesta, entrenamiento y registro.

2. Infraestructura reproducible con MLflow

2.1. Tracking server y almacenamiento

El servidor de tracking se ejecuta localmente en `127.0.0.1:5000`. MLflow permite utilizar SQLite como backend para conservar metadatos de corridas, métricas, parámetros, tags y modelos; además, el cliente puede conectarse al servidor mediante la variable `MLFLOW_TRACKING_URI` (MLflow, s.f.-a, s.f.-d).

```
mlflow server \
```

```
  --backend-store-uri sqlite:///mlflow.db \
  --host 127.0.0.1 \
  --port 5000


export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
```

La base `mlflow.db` se utiliza como backend local, mientras que los artefactos permanecen fuera del historial de Git. Esta separación permite conservar trazabilidad experimental sin convertir el repositorio de código en un almacén de datos o modelos.

2.2. MLflow Project

El archivo `MLproject` define tres entrypoints reproducibles.

Tabla 1
Entrypoints definidos en MLflow Project.

Entrypoint	Función	Parámetros principales
`ingest`	Ejecuta la ingesta y generación del manifiesto de datos.	Sin parámetros obligatorios.
`train`	Ejecuta baseline, candidato o búsqueda de una pregunta.	`question`, `model`, `params_json`.
`register`	Registra los tres modelos seleccionados y asigna alias.	Configuración de modelos seleccionados.

El archivo `MLproject` utiliza `python_env.yaml` para definir el entorno reproducible del proyecto. La instalación de dependencias, la ingesta y la ejecución de los entrypoints fueron verificadas desde un clon limpio del repositorio, de forma que el flujo no dependa de configuraciones locales no documentadas.

La ejecución mínima desde un entorno preparado es:

```
mlflow run . -e ingest --env-manager local
```

Para reproducir los modelos seleccionados:

```
mlflow run . --experiment-name mibici-equipo32 \
  -e train -P question=p1 -P model=poisson \
  -P params_json='{}' --env-manager local

mlflow run . --experiment-name mibici-equipo32 \
  -e train -P question=p2 -P model=hgb \
  -P params_json='{"learning_rate":0.05,"max_iter":350,
  "max_leaf_nodes":63,"min_samples_leaf":30,
  "l2_regularization":2.0}' --env-manager local

mlflow run . --experiment-name mibici-equipo32 \
  -e train -P question=p3 -P model=kmeans_k4 \
  -P params_json='{}' --env-manager local
```

El uso de `mlflow run` evita depender de ejecuciones manuales aisladas y hace explícitos los parámetros utilizados en cada experimento (MLflow, s.f.-c).

2.3. Particiones temporales

La estrategia de validación se mantiene constante entre baseline y candidatos.

Tabla 2
Particiones temporales utilizadas en la Etapa 2.

Pregunta	TRAIN	VALIDATION / criterio de selección	TEST
P1	Enero–septiembre de 2025	Octubre de 2025	Noviembre–diciembre de 2025
P2	Enero–septiembre de 2025	Octubre de 2025	Noviembre–diciembre de 2025
P3	1 ene.–19 oct. de 2025	Métricas internas sobre TRAIN; búsqueda de k	20 oct.–31 dic. de 2025

Para P3 también se utilizó una ventana histórica reducida del 1 de abril al 19 de octubre de 2025 para evaluar estabilidad. En P1 y P2 la selección se realizó sobre VALIDATION. En P3 la búsqueda de k se realizó con `train_silhouette` como métrica principal y `train_davies_bouldin` como secundaria. TEST permaneció fuera del proceso de selección en las tres preguntas.

3. Tracking avanzado, búsquedas y artefactos

3.1. Convención de tracking

Las corridas se registran en el experimento `mibici-equipo32`. Cada candidato conserva parámetros, métricas y tags de trazabilidad, entre ellos `question`, `model_family`, `stage_of_work`, `team`, `git_commit` y `data_version`. MLflow Tracking está orientado precisamente a conservar metadatos, parámetros, métricas y artefactos de cada run (MLflow, s.f.-a; Zaharia et al., 2018).

La versión de datos final es común a los modelos seleccionados:

```
data_version = 7aa5e436d887
```

3.2. Búsquedas con nested runs

Cada búsqueda de hiperparámetros se organizó mediante un parent run y un child run por combinación evaluada.

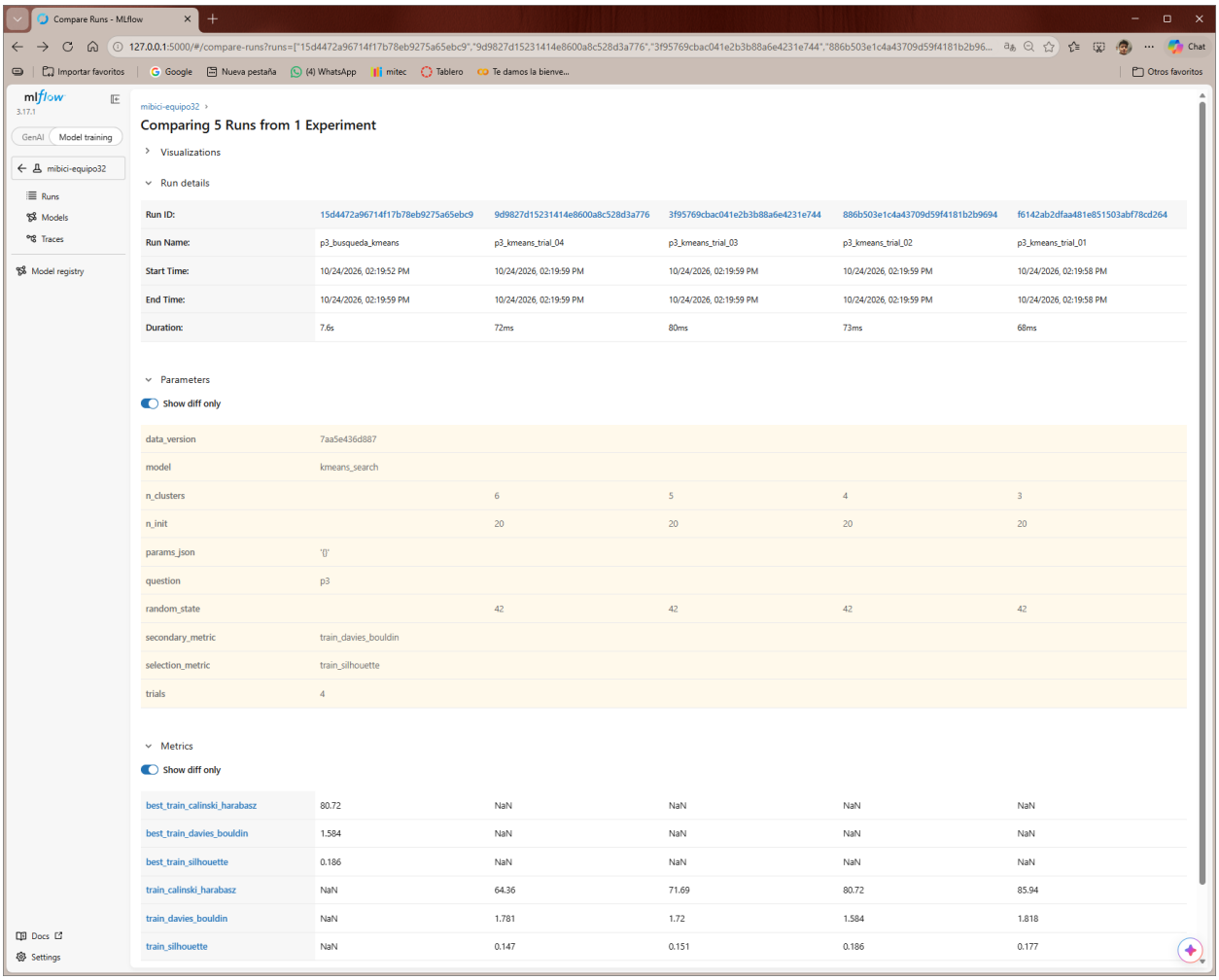
Tabla 3
Búsquedas de hiperparámetros registradas como nested runs.

Pregunta	Parent run	Run ID	Criterio
P1	`p1_busqueda_hgb`	`73ec2d7bf4a541fc8a6b77[illegible]`	[illegible]mae.
P2	`p2_busqueda_hgb`	`ced46cb04c204f7096d9cc[illegible]`	[illegible]; PR-AUC secundaria.
P3	`p3_busqueda_kmeans`	`15d4472a96714f17b78eb9[illegible]`	[illegible]; Davies–Bouldin secundario.

En P3 se evaluaron $k = 3, 4, 5, 6$ con `n_init=20`; la mejor combinación de la búsqueda fue $k = 4$, con Silhouette TRAIN de 0.1864 y Davies–Bouldin TRAIN de 1.5841.

Figura 1

Comparación del parent run de P3 y sus cuatro child runs en MLflow. La vista muestra los valores de k, las métricas de selección y la estructura de la búsqueda.



3.3. Artefactos por candidato

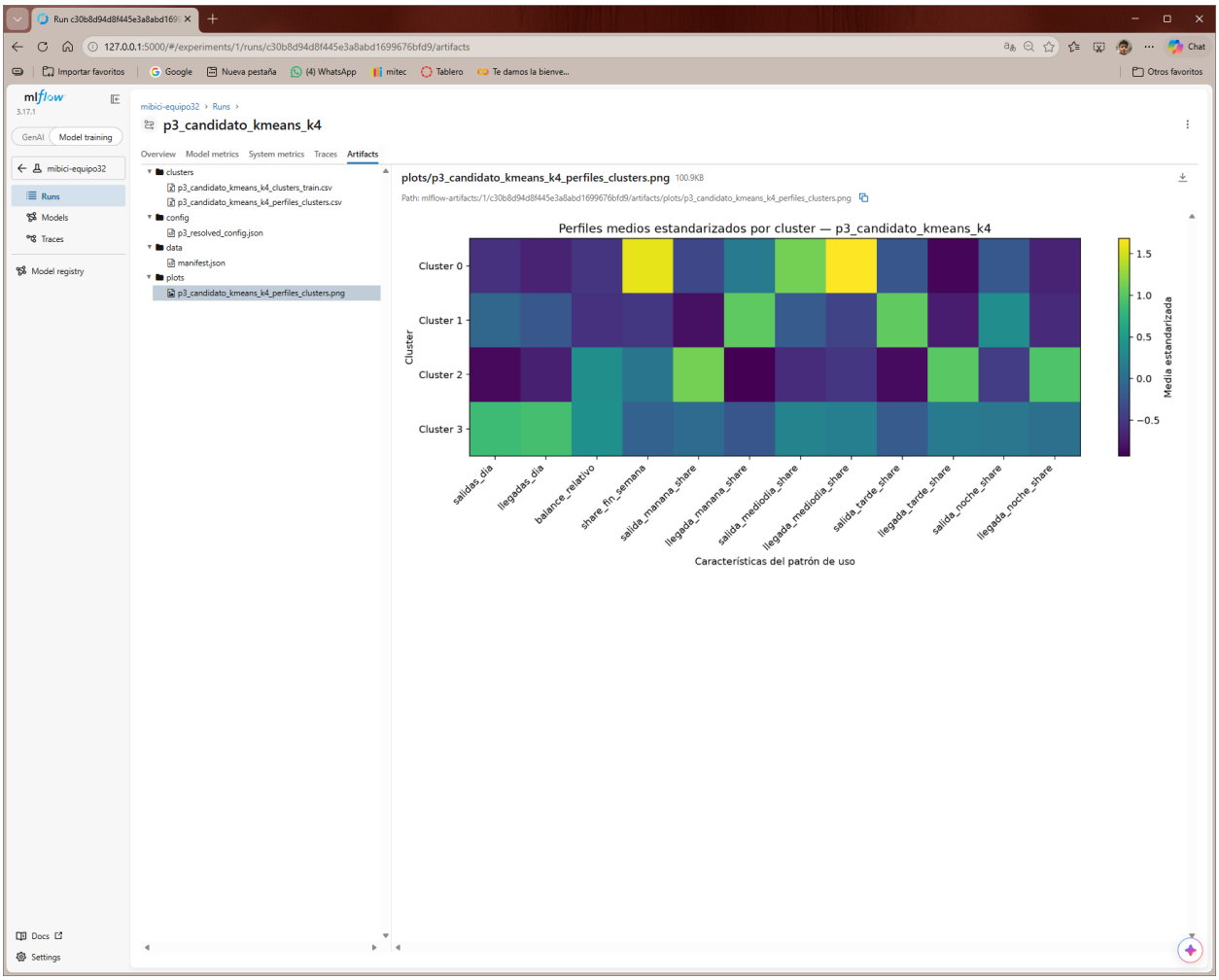
Los candidatos finales registran el modelo mediante `log_model`, firma, ejemplo de entrada, configuración resuelta, manifiesto de datos y artefactos de evaluación. El almacenamiento de artefactos junto con los metadatos del run permite relacionar el resultado con la evidencia que lo respalda (MLflow, s.f.-a).

Tabla 4
Artefactos principales registrados por pregunta.

Pregunta	Evidencia registrada
P1	Modelo con firma y ejemplo de entrada (*input example*), configuración, manifiesto, métricas, predicciones, MAE por día y comparación de demanda real frente a predicha.
P2	Modelo con firma y ejemplo de entrada (*input example*), configuración, manifiesto, matriz de confusión, curva precision–recall y reporte de clasificación.
P3	Modelo con firma y ejemplo de entrada (*input example*), configuración, manifiesto, asignaciones de cluster, perfiles medios por cluster y visualización de perfiles estandarizados.

Figura 2

Artefactos del candidato final de P3. La estructura muestra carpetas de clusters, configuración, manifiesto y visualización de perfiles.



Los artefactos registrados en MLflow no incluyen atributos personales de los usuarios. El pipeline conserva únicamente las variables necesarias para las tres preguntas analíticas y evita almacenar información que no aporta al modelado.

4. P1: predicción de demanda

P1 estima el número de viajes que se iniciarán en cada estación al día siguiente. La variable objetivo es un conteo no negativo, por lo que PoissonRegressor constituye una alternativa coherente

con la naturaleza de la respuesta. Los modelos lineales generalizados con distribución Poisson y enlace logarítmico son apropiados para objetivos de conteo (scikit-learn developers, s.f.-c).

Se comparó un baseline semanal con cuatro candidatos. MAE fue la métrica principal de selección, complementada con RMSE, WAPE y R^2.

Tabla 5
Comparación final de modelos para P1.

Modelo	Configuración clave	VAL MAE ↓	TEST MAE ↓	Mejora VAL vs. baseline
Baseline semanal	Rezago de 7 días	6.2316	7.2195	Referencia
PoissonRegressor	`alpha=0.001`, `max_iter=1000`	**5.1038**	**5.4098**	**18.1 %**
HGB Poisson	`lr=0.06`, `iter=400`, `leaf=31`	5.3264	6.3395	14.5 %
Random Forest	`n=200`, `depth=16`, `leaf=2`	5.1040	5.6379	18.1 %
HGB tuneado	`lr=0.06`, `iter=500`, `leaf=63`, `L2=2`	5.2205	6.0814	16.2 %

El PoissonRegressor seleccionado utiliza `alpha=0.001` y `max_iter=1000`. Frente al baseline reduce el MAE en 18.1 % sobre VALIDATION. Además, obtuvo el menor MAE de validación entre los modelos comparados, con 5.1038, aunque la diferencia respecto a Random Forest fue marginal. Ante desempeños prácticamente equivalentes, se privilegió la adecuación del modelo a una variable objetivo de conteo y su menor complejidad operativa. Posteriormente, TEST se utilizó únicamente como evaluación final y confirmó la capacidad de generalización del modelo, con un MAE de 5.4098 y un WAPE de 0.1753.

Selección P1

Se seleccionó **PoissonRegressor**. La decisión se tomó con base en VALIDATION, donde obtuvo el menor MAE entre los candidatos, complementando este resultado con su adecuación a una respuesta de conteo y su menor complejidad operativa. TEST se reservó para evaluación final. El run seleccionado es `371bf2379bfa4a74b8effbe02d2ee3ae`.

El modelo quedó logueado en MLflow con URI `models:/m-f1e2eda1d16a4663a7ec1f50d75859d9`, firma y ejemplo de entrada (*input example*). Su commit de origen es `e02f9450d245390024f328467f0732b98e200`

5. P2: clasificación de riesgo

P2 clasifica si una estación estará en riesgo de quedarse sin bicicletas o sin anclajes en las próximas dos horas. Debido al desbalance y al costo operativo de omitir estaciones riesgosas, la selección utiliza F1 como métrica principal y PR-AUC como criterio secundario. El umbral final, 0.877356, se determinó únicamente con VALIDATION.

Se evaluaron Logistic Regression balanceada como baseline, HistGradientBoostingClassifier, Random Forest y un HistGradientBoostingClassifier tuneado. El estimador de gradient boosting basado en histogramas está diseñado para clasificación y permite controlar parámetros como learning rate, número de iteraciones, hojas máximas y regularización L2 (scikit-learn developers, s.f.-b).

Tabla 6
Comparación final de modelos para P2.

Modelo	Configuración clave	VAL F1 ↑	TEST F1 ↑	Mejora VAL vs. baseline
Logistic Regression	`class_weight=balanced`	0.4151	0.3645	Referencia
HGB	Configuración base	0.4680	**0.4009**	12.7 %
Random Forest	Configuración base	0.4555	0.3856	9.7 %
HGB tuneado	`lr=0.05`, `iter=350`, `leaf=63`, `L2=2`	**0.4691**	0.4007	**13.0 %**

El HGB tuneado mejora 13.0 % el F1 de VALIDATION respecto al baseline y alcanza además el mayor PR-AUC de VALIDATION, con 0.4869 frente a 0.4146 del baseline, equivalente a una mejora aproximada de 17.4 %. En TEST obtiene F1 de 0.4007, PR-AUC de 0.3994, recall de 0.4849 y precisión de 0.3414.

El HGB sin tuning presenta un TEST F1 marginalmente mayor, con 0.4009 frente a 0.4007. Sin embargo, esta diferencia no modifica la selección, ya que TEST se reservó exclusivamente para evaluación final. El HGB tuneado conserva la mejor combinación de F1 y PR-AUC en

VALIDATION.

Selección P2

Se seleccionó **HistGradientBoostingClassifier tuneado** con `learning_rate=0.05`, `max_iter=350`, `max_leaf_nodes=63`, `min_samples_leaf=30` y `l2_regularization=2.0`. El run seleccionado es `9e6d7264d7a74a68a9f8a02f4be3b03f`.

Aunque el HGB tuneado implica una complejidad mayor que el baseline lineal, la mejora obtenida en F1 y PR-AUC justificó ese costo adicional dentro del contexto de la tarea.

El modelo quedó logueado con URI `models:/m-8bddd02b531d4d61936d74a3f9cadc93`. Su commit de origen es `99cfd3e9473bf3614caefc9d91d2546f5ba20606`.

6. P3: segmentación de estaciones

P3 agrupa estaciones de acuerdo con 12 características de intensidad, balance y distribución temporal de llegadas y salidas. K-Means requiere especificar el número de clusters y optimiza una partición alrededor de centroides (scikit-learn developers, s.f.-a). La evaluación combina calidad interna con estabilidad temporal.

Para interpretar la calidad de la partición se utilizaron Silhouette y Davies–Bouldin. Silhouette aumenta cuando los grupos son compactos y están separados; en Davies–Bouldin valores menores representan una mejor separación relativa (scikit-learn developers, s.f.-a). ARI y NMI se utilizaron para medir acuerdo entre asignaciones de diferentes periodos.

Tabla 7
Comparación final de modelos para P3.

Modelo	Configuración clave	TRAIN Silhouette ↑	TRAIN D-B ↓	Mejora D-B vs. baseline
K-Means $k = 2$	`n_clusters=2`	**0.1977**	1.7715	Referencia
K-Means $k = 3$	`n_clusters=3`, `n_init=20`	0.1773	1.8178	−2.6 %
K-Means $k = 4$	`n_clusters=4`, `n_init=20`	0.1864	**1.5841**	**10.6 %**
K-Means $k = 5$	`n_clusters=5`, `n_init=20`	0.1508	1.7201	2.9 %

Una vez seleccionado $k = 4$, TEST se utilizó únicamente para evaluar generalización temporal. En este periodo el modelo obtuvo Silhouette de 0.1666, Davies–Bouldin de 1.7448, ARI de 0.7927 y NMI de 0.7361.

Como comparación adicional se evaluó Agglomerative Clustering con $k = 4$, que obtuvo un Silhouette de TRAIN de 0.1090 y un Davies–Bouldin de 1.9998. Esta alternativa se mantuvo como `candidate_comparison` y no como candidato desplegable, debido a que no proporciona una operación `predict` equivalente a K-Means para asignar nuevas observaciones de manera directa.

El baseline con $k = 2$ obtuvo el mayor valor de Silhouette; sin embargo, la selección de $k = 4$ no se basó exclusivamente en esta métrica. La búsqueda de hiperparámetros realizada sobre TRAIN identificó a $k = 4$ como la configuración con mejor equilibrio entre Silhouette y Davies–Bouldin, mientras que la evaluación frente a la ventana histórica reducida mostró una alta estabilidad, con ARI de 0.9415 y NMI de 0.9167. Además, una solución de cuatro grupos ofrece una mayor granularidad para diferenciar patrones de uso entre estaciones.

Una vez realizada la selección, TEST se utilizó únicamente como evaluación final. En este conjunto, $k = 4$ obtuvo un Davies–Bouldin de 1.7448, un ARI train-test de 0.7927 y un NMI train-test de 0.7361, resultados que respaldan la estabilidad temporal del modelo sin haber intervenido en su selección.

Selección P3

Se seleccionó **K-Means con** $k = 4$ **y** `n_init=20` a partir de la búsqueda realizada sobre TRAIN, considerando calidad interna, estabilidad frente a la ventana histórica reducida y granularidad de la solución. TEST se reservó exclusivamente para verificar posteriormente la estabilidad temporal del modelo seleccionado. El run final utilizado para Registry es `c30b8d94d8f445e3a8abd1699676bfd9`.

Desde el punto de vista operativo, K-Means conserva además una asignación directa de nuevas observaciones mediante centroides, lo que favorece su uso frente a alternativas sin una operación `predict` equivalente.

El modelo quedó logueado con URI `models:/m-7918112016fa487f87ac4e87eb31d359`. Su commit de origen es `ff1bdba3a500d296a0507451c1674d0ba16f4f8a`.

7. Model Registry y ciclo de vida

MLflow Model Registry centraliza versiones, aliases, tags y linaje hacia el experimento y run que produjo cada modelo (MLflow, s.f.-b). Para esta entrega se utilizó un esquema basado en aliases y se reservó `champion` para identificar la versión seleccionada como referencia productiva. MLflow permite cargar una versión mediante una URI como `models:/nombre@champion`, y el alias puede reasignarse posteriormente sin modificar a los consumidores del modelo (MLflow, s.f.-b).

Tabla 8
Modelos registrados y esquema de ciclo de vida.

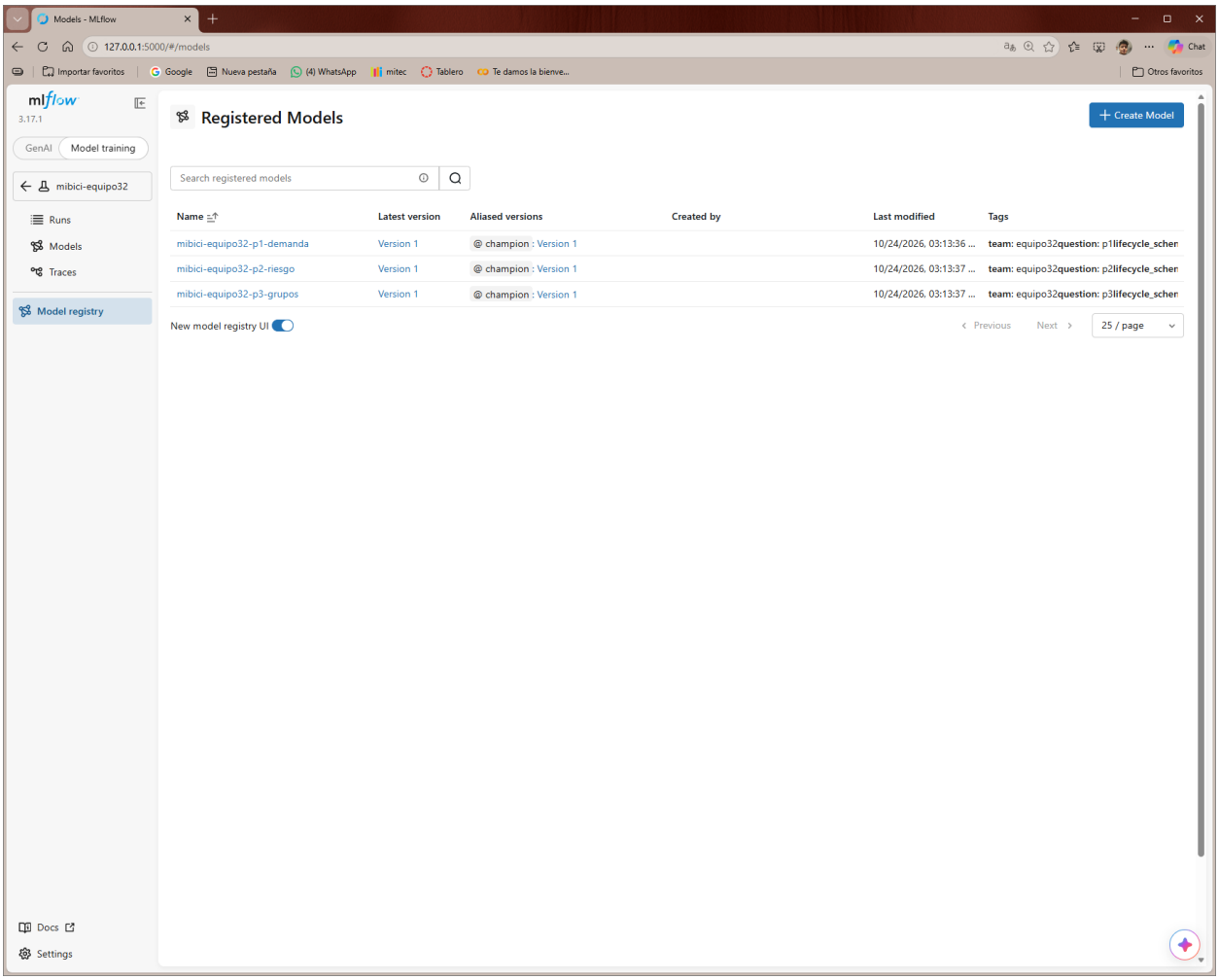
Pregunta	Registered Model	Versión	Alias	Source run
P1	`mibici-equipo32-p1-demanda`	1	`champion`	`371bf2379bfa4a74b8effbe02d2ee3ae`
P2	`mibici-equipo32-p2-riesgo`	1	`champion`	`9e6d7264d7a74a68a9f8a02f4be3b03f`
P3	`mibici-equipo32-p3-grupos`	1	`champion`	`c30b8d94d8f445e3a8abd1699676bfd9`

El entrypoint reproducible para registrar o verificar los modelos es:

```
mlflow run . --experiment-name mibici-equipo32 \
  -e register --env-manager local
```

Figura 3

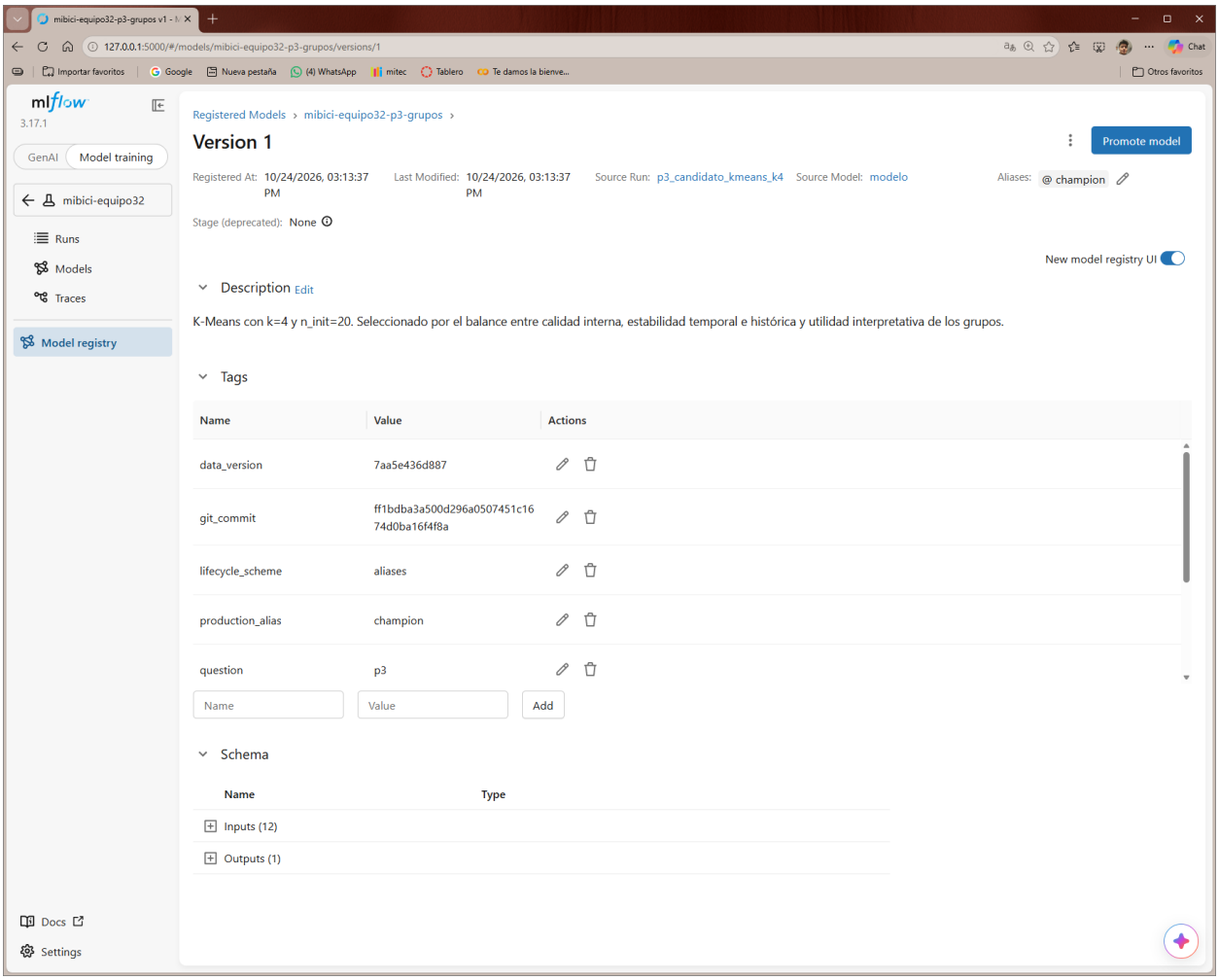
Model Registry con los tres modelos registrados como versión 1 y alias `champion`.



La versión de P3 muestra de forma explícita el source run, alias, `data_version`, `git_commit`, `lifecycle_scheme`, `production_alias` y pregunta asociada. La firma registrada contiene 12 entradas y una salida.

Figura 4

Linaje de la versión 1 del modelo `mibici-equipo32-p3-grupos`*. Se observan el alias* `champion`*, source run y tags de trazabilidad.*



Las URIs de carga verificadas son:

```
models:/mibici-equipo32-p1-demanda@champion
models:/mibici-equipo32-p2-riesgo@champion
models:/mibici-equipo32-p3-grupos@champion
```

Los tres modelos fueron cargados correctamente como `PyFuncModel`. El esquema permite que una versión futura se registre como una nueva versión, se evalúe y, solo después de aprobarse, reciba el alias `champion`. De este modo el alias representa una referencia mutable y controlada,

mientras que cada versión conserva identidad y linaje propios.

Los tres modelos registrados conservan trazabilidad hacia su *source run*, versión de datos y commit de Git. La Figura 4 muestra el detalle de P3 como ejemplo del esquema de linaje aplicado de manera consistente a P1, P2 y P3.

8. Git, repositorio y reproducibilidad

El desarrollo siguió un flujo basado en ramas y Pull Requests. GitHub flow propone precisamente trabajar en ramas separadas, integrar cambios mediante Pull Requests y conservar el historial asociado (GitHub, s.f.). En esta versión, las funcionalidades se integraron de forma incremental.

Tabla 9
Integración de la Etapa 2 mediante Pull Requests.

PR	Commit en `main`	Alcance
#1	`251e57e`	Infraestructura reproducible con MLflow Projects.
#2	`9c136ee`	Experimentación avanzada y tuning de P1.
#3	`17308c3`	Experimentación avanzada y tuning de P2.
#4	`b7a7a24`	Experimentación reproducible y tuning de P3.
#5	`b41cbde`	Artefactos visuales y tercer candidato completo de P3.
#6	`85fe07c`	Registro reproducible en MLflow Model Registry.
#7	`a30792d`	README definitivo para la segunda entrega.

La reproducción desde un clon limpio comienza con:

```
git clone \
https://github.com/GIvanConder/TC5061-Proyecto-MiBici-Gerardo-Ivan-
   Conde-Reynoso.git
```

```
cd TC5061-Proyecto-MiBici-Gerardo-Ivan-Conde-Reynoso
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pip check
```

A continuación se inicia el tracking server, se configura `MLFLOW_TRACKING_URI` y se ejecutan los entrypoints del `MLproject`. El README documenta este procedimiento y los modelos seleccionados.

La auditoría final de Git confirmó que no se encuentran versionados datos raw, interim, processed o external; tampoco `mlflow.db`, `mlruns/`, `mlartifacts/`, archivos Parquet, CSV de resultados o modelos `.pkl/.joblib`. Esta separación reduce acoplamiento entre código y artefactos generados, un problema frecuente en sistemas de aprendizaje automático mantenidos a lo largo del tiempo (Sculley et al., 2015).

8.1. Condición de trabajo individual

Por una situación particular del equipo, el personal docente autorizó que cada integrante desarrollara y entregara su propia versión del proyecto, utilizando una rama o repositorio independiente e indicando esta condición en Canvas. En consecuencia, esta entrega corresponde a la versión individual de Gerardo Iván Conde Reynoso.

La evidencia de control de versiones se conserva mediante ramas, commits y Pull Requests del repositorio individual. No se simulan revisores ni colaboraciones inexistentes. Esta decisión mantiene la trazabilidad real del trabajo y documenta de forma transparente la excepción autorizada.

9. Conclusiones

La Etapa 2 permitió transformar el proyecto MiBici en un flujo experimental con mayor control sobre reproducibilidad, trazabilidad y ciclo de vida. MLflow Projects estandarizó los entrypoints de ingesta, entrenamiento y registro; MLflow Tracking concentró parámetros, métricas, tags y artefactos; las búsquedas con nested runs permitieron organizar de forma clara los procesos de tuning; y Model Registry vinculó los modelos seleccionados con sus corridas de origen, commits de Git y versión de datos.

Los resultados mostraron también que la selección de un modelo no debe depender únicamente de su complejidad ni del mejor valor observado en TEST. En P1, PoissonRegressor superó al baseline y mantuvo un desempeño competitivo frente a modelos más complejos, con una reducción aproximada de 25.1 % en MAE sobre TEST. En P2, el HistGradientBoostingClassifier tuneado fue seleccionado con base en VALIDATION, aunque otro candidato obtuvo posteriormente una diferencia mínima a favor en TEST F1. Esto permitió conservar la independencia del conjunto de evaluación final y evitar una selección retrospectiva.

P3 presentó una situación diferente, ya que la calidad de un clustering no puede resumirse adecuadamente con una sola métrica. Aunque el baseline con $k = 2$ obtuvo el mayor Silhouette, la solución con $k = 4$ mostró un mejor equilibrio entre Silhouette y Davies–Bouldin durante la búsqueda realizada sobre TRAIN, además de una alta estabilidad frente a la ventana histórica reducida. Por ello, su selección se sustentó en un criterio multicriterio de calidad interna, estabilidad y granularidad de los grupos. Posteriormente, TEST confirmó la estabilidad temporal del modelo mediante sus valores de ARI y NMI, sin intervenir en la selección.

Otro avance importante fue la incorporación de trazabilidad mediante `data_version`, `git_commit`, configuraciones resueltas, firmas de modelo y artefactos. Esta información permite relacionar cada resultado con una versión concreta de los datos y del código, haciendo posible reconstruir las condiciones bajo las cuales se obtuvo cada modelo.

Finalmente, el uso de MLflow Model Registry permitió registrar los tres modelos seleccionados como versión 1 y asociarlos al alias `champion`. Este esquema separa una versión específica

del modelo de su función dentro del ciclo de vida y facilita que futuras iteraciones puedan evaluarse antes de sustituir al modelo vigente.

En conjunto, el principal resultado de esta etapa no fue únicamente mejorar los modelos respecto a sus baselines, sino construir una relación verificable entre datos, código, experimentos, resultados y modelos registrados. Esta integración representa el principal avance de MLOps alcanzado en el proyecto y permite que las decisiones tomadas sean más reproducibles, trazables y fáciles de revisar.

Contrato de propiedad académica y uso responsable de inteligencia artificial

Durante el desarrollo de esta etapa se utilizaron herramientas de inteligencia artificial generativa como apoyo en tareas de revisión de redacción, organización del reporte, contraste metodológico, depuración de código y documentación técnica. Su uso se limitó a funciones auxiliares y no sustituyó la ejecución de experimentos, la inspección de corridas, la validación de métricas ni la selección de evidencia.

Las salidas de herramientas de inteligencia artificial no se consideraron evidencia experimental por sí mismas. Las métricas, identificadores de runs, artefactos, versiones registradas y capturas presentadas en este documento provienen de la ejecución del proyecto y fueron contrastadas con MLflow, Git y los archivos del repositorio.

El autor conserva la responsabilidad académica sobre las decisiones metodológicas, implementación, resultados, interpretación y referencias utilizadas. Las herramientas de inteligencia artificial se emplearon como apoyo de productividad y revisión, no para fabricar resultados, alterar evidencia o sustituir la trazabilidad proporcionada por Git, `data/manifest.json` y MLflow.

Declaración de responsabilidad

La versión final del documento corresponde a una entrega individual autorizada. Gerardo Iván Conde Reynoso asume la responsabilidad de validar el contenido, los resultados y las evidencias incorporadas antes de su entrega.

Referencias

GitHub. (s.f.). *GitHub flow*. Consultado el 24 de octubre de 2026, desde https://docs.github.com/en/get-started/using-github/github-flow

MiBici. (s.f.). *Datos abiertos*. Consultado el 24 de octubre de 2026, desde https://www.mibici.net/es/datos-abiertos/

MLflow. (s.f.-a). *ML Experiment Tracking*. Consultado el 24 de octubre de 2026, desde https://mlflow.org/docs/latest/tracking/

MLflow. (s.f.-b). *ML Model Registry*. Consultado el 24 de octubre de 2026, desde https://mlflow.org/docs/latest/model-registry/

MLflow. (s.f.-c). *MLflow Projects*. Consultado el 24 de octubre de 2026, desde https://mlflow.org/docs/latest/projects/

MLflow. (s.f.-d). *MLflow Tracking Server*. Consultado el 24 de octubre de 2026, desde https://mlflow.org/docs/latest/tracking/server/

scikit-learn developers. (s.f.-a). *Clustering*. Consultado el 24 de octubre de 2026, desde https://scikit-learn.org/stable/modules/clustering.html

scikit-learn developers. (s.f.-b). *HistGradientBoostingClassifier*. Consultado el 24 de octubre de 2026, desde https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html

scikit-learn developers. (s.f.-c). *Linear Models*. Consultado el 24 de octubre de 2026, desde https://scikit-learn.org/stable/modules/linear_model.html

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, *28*. https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems

Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the

Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin*, *41*(4), 39-45. https://people.eecs.berkeley.edu/~matei/papers/2018/ieee_mlflow.pdf